

Advanced Parallelization Strategies for Neural Polar Decoders on the Two-User Multiple Access Channel

The development of polar codes, since their inception by Arikan in 2009, represents a landmark achievement in information theory, providing the first explicit construction of codes that provably achieve the capacity of binary-input memoryless symmetric channels with $O(N \log N)$ encoding and decoding complexity.¹ The extension of this paradigm to the two-user Multiple Access Channel (MAC) by Sasoglu et al. in 2013 demonstrated that polarization remains a viable mechanism for achieving the entire capacity region of multi-user systems.¹ However, the practical deployment of these codes is constrained by the sequential nature of the Successive Cancellation (SC) decoding algorithm, which presents a significant latency bottleneck in high-throughput communications.⁵ While the single-user Neural Polar Decoder (NPD) has recently achieved a breakthrough in training efficiency through the "fast cross-entropy" (fast_ce) parallelization trick, which reduces sequential training depth from $O(N \log N)$ to $O(\log N)$, this technique does not directly extend to the multi-user MAC setting due to fundamental structural and mathematical divergences.¹ The search for a parallelizable training decomposition for neural MAC polar decoders remains an open and critical research frontier, requiring a synthesis of algebraic group theory, tropical geometry, and advanced neural architectures.¹

The Evolution of Sequential Depth in Polar Decoding

The foundational single-user SC decoder operates on a binary tree structure of depth $n = \log_2 N$, where N is the block length.¹ At each internal node, the decoder performs check-node (f-node) and variable-node (g-node) operations.¹ The f-node operation involves an LLR-based calculation that is independent of other nodes at the same tree depth, while the g-node operation depends on the hard bit decision propagated from the f-node.¹ In a standard sequential tree walk, the decoder visits $2N - 1$ nodes, resulting in a latency that grows linearly with the block length.⁶

The Neural Polar Decoder (NPD), introduced by Aharoni et al. (2024), replaces these analytical operations with multi-layer perceptrons (MLPs).¹ The most significant innovation of the NPD is the fast_ce paradigm, which exploits the availability of true bits during teacher-forced training.¹ Because the true bits are known, the dependency of g-nodes on previous decisions is effectively resolved during the training phase, allowing all f-nodes and g-nodes at a given tree level to be

computed in parallel.¹ This reduces the sequential gradient depth to exactly $\log_2 N$ steps, making training feasible for large block lengths such as $N = 1024$.¹ The two-user MAC problem, however, introduces a qualitative shift in complexity. Instead of scalar LLRs, the MAC decoder propagates 2×2 joint probability tensors representing the distribution of bits from User U and User V.¹ The operations at each node—CalcLeft, CalcRight, and CalcParent—are mathematically more demanding.¹ The CalcLeft operation is a circular convolution over the Klein four-group ($Z_2 \times Z_2$), which couples all four elements of the tensor.¹ Furthermore, the Class B decoding path, which is essential for achieving symmetric rates ($R_u \approx R_v$), requires a bottom-up CalcParent operation to combine information from subtrees.¹ This non-top-down information flow creates vertical dependencies that are absent in the single-user case, preventing the simple level-alignment required for fast_ce.¹

Mathematical Barriers to Level-Parallelism in MAC Decoders

The primary obstacle to parallelizing the training of a neural MAC decoder is the non-separability of the circular convolution in the log-probability domain.¹ In the probability domain, circular convolution over the Klein four-group can be diagonalized using the Walsh-Hadamard Transform (WHT).¹ Specifically, if P_{parent} and P_{right} are the input probability tensors, their circular convolution P_{left} satisfies:

$$\text{WHT}(P_{left}) = \text{WHT}(P_{parent}) \cdot \text{WHT}(P_{right})$$

where the multiplication is element-wise.¹ In this domain, the operation is perfectly parallel and decomposable.¹

However, neural decoders must operate in the log-domain to maintain numerical stability and avoid the exponential precision issues associated with multiplying many small probabilities.⁹ In the log-domain, the multiplication of probabilities becomes simple addition, which is parallelizable.⁹ However, the WHT involves sums of probabilities, which in the log-domain translate to the LogSumExp (LSE) operation.¹ The LSE function is defined as:

$$\text{LSE}(x_1, x_2, \dots, x_n) = \log \sum_{i=1}^n \exp(x_i)$$

This operation is fundamentally non-separable.¹ Any attempt to perform the circular convolution via the WHT in the log-domain would require a sequence of

$\exp \rightarrow \text{WHT} \rightarrow \text{element-wise addition} \rightarrow \text{inverse WHT} \rightarrow \log$.¹ The neural network, when tasked with learning this sequence, encounters an optimization plateau because the coupling introduced by the LSE sums across the 4-element tensor prevents the level-wise independence that makes single-user parallelization possible.¹

Empirical evidence from failed approaches underscores this challenge. Attempts to train decoders in the WHT domain or using sign-encodings similar to NPD consistently plateau at a loss level around 0.30, which is near-random for a 4-class classification problem.¹ This indicates

that the neural representation cannot easily "bypass" the non-linear coupling of the $Z_2 \times Z_2$ group structure when forced into a parallel framework that does not respect the underlying group convolution.¹

Structural Dependencies and the CalcParent Challenge

Beyond the mathematical coupling of the tensor elements, the MAC decoder faces a structural dependency problem related to the decoding path.¹ Single-user decoders follow a rigid,

level-aligned pattern where bits are decided sequentially from index 0 to $N - 1$.¹ In the MAC setting, the decoding order is governed by a monotone chain path.¹ While Class A (decoding all of User V then all of User U) and Class C (decoding all of User U then all of User V) reduce to two independent single-user decoders, they cannot achieve the symmetric rate point.¹

The Class B path, which interleaves User U and User V decisions to achieve symmetric rates ($R_u \approx R_v \approx 0.48$), is the most practically relevant but the hardest to parallelize.¹ The

interleaved order means the decoder must frequently navigate "up" the tree using the CalcParent operation to provide the necessary context for the next decision.¹ CalcParent is essentially a marginalization operation—the inverse of circular convolution—that combines the left and right child tensors to update the parent edge.¹ This creates a bottom-up dependency where the state of a parent node depends on the results of both subtrees, violating the strictly

top-down information flow required for $O(\log N)$ depth training.¹

The complexity of these operations is reflected in the total number of computations required per codeword. For a block length $N = 128$ under the Class B path, the decoder performs approximately 750 tensor operations.¹

Operation Type	Formulaic Nature	Frequency (approx.)
CalcLeft	$\sum_{u',v'} P_p[u \oplus u', v \oplus v'] \cdot$	$2N$
CalcRight	$P_p[d_l \oplus u, d_v \oplus v] \cdot P_l[d_l, d]$	$2N$
CalcParent	$\sum_{u',v'} P_l[u', v'] \cdot P_r[u \oplus u', v \oplus v']$	$2N - 2$

The sequential accumulation of these operations results in an $O(N \log N)$ training depth.¹

For $N = 1024$, this corresponds to over 6,000 sequential operations, creating an optimization barrier characterized by vanishing gradients and extreme sequential error accumulation.¹

Strategic Directions for Training Parallelization

To reduce the training depth to $O(\log N)$ or $O(N)$, several innovative research directions must be explored, each targeting a specific aspect of the coupling or dependency issues described above.¹

Algebraic Decomposition and Character Domain Representations

The first direction involves a fundamental shift in how the neural decoder represents joint probabilities.¹ Since circular convolution is diagonalized by the WHT, it is possible that a neural

representation operating natively in the "frequency" or "character" domain of the $Z_2 \times Z_2$ group could resolve the coupling issue.¹ The Klein four-group has four distinct characters, and in this space, the joint convolution reduces to simple element-wise multiplication.¹

The critical research question is whether the z-encoder (which maps channel observations to embeddings) can learn to output embeddings that are already in this frequency domain.¹ If the z-encoder can perform this transformation, then the CalcLeft operation becomes a simple element-wise MLP update, which is fully parallelizable.¹ The challenge remains for CalcRight and CalcParent, as these are not circular convolutions and may not have simple element-wise forms in the WHT domain.¹ However, the success of this approach would reduce the sequential depth of the CalcLeft-heavy portion of the tree to $O(\log N)$, matching the single-user performance.¹

Tropical Algebra and the Max-Log Transformation

The "max-log" approximation, widely used in the decoding of LDPC and convolutional codes, offers another pathway to parallelization.¹ By replacing the LogSumExp operation with the

maximum operator ($\text{LSE}(a, b) \approx \max(a, b)$), the neural decoder transitions from a smooth, non-separable space to a piecewise-linear tropical semiring.¹

In the tropical semiring ($\max, +$), addition is replaced by $\max$ and multiplication is replaced by addition.⁹ Research into polar code decoding under the min-sum approximation has shown that strong polarization still holds, suggesting that the fundamental information-theoretic properties of the code are preserved.¹² If the circular convolution can be shown to be decomposable into independent per-element operations under this max-plus algebra, it would enable a level-parallel training scheme.¹ The neural network would then only need to learn the small

"correction terms" required to bridge the gap between the max-log approximation and the true LSE result, a task that has proven successful in single-user NPD architectures.¹

Factored Belief Propagation and Factor Graph Parallelization

A more radical departure from the SC tree walk is the adoption of a factor graph representation of the MAC polar code.¹ Belief Propagation (BP) algorithms are inherently parallelizable because all message updates at a given iteration can be performed simultaneously.¹ For a polar

code of block length N , the factor graph has $n = \log_2 N$ stages, and messages can be updated in a parallel "flooding" schedule.¹⁵

A neural version of this algorithm, such as weighted BP with learned parameters, would have a sequential training depth of $O(\text{iterations})$, which is independent of N if the number of iterations required for convergence is constant.¹ Nachmani et al. (2018) have demonstrated that learned weights on the edges of the Tanner graph can significantly improve the performance of BP decoders.¹ Adapting this to the two-user MAC would involve creating joint variable and

check nodes for the $\mathbb{Z}_2 \times \mathbb{Z}_2$ constraints.¹⁹ While BP generally suffers from higher error rates than SCL for finite code lengths, the neural weights and potential "joint iterative detection and decoding" (JIDD) schemes could close this gap, providing a path to high-speed, parallel training.¹⁵

Transformer-Based Sequence Mapping

Transformers, with their self-attention mechanisms, process entire sequences in parallel with $O(1)$ sequential depth per layer.¹ A transformer-based parallel decoder for the MAC could

theoretically take all N channel observations as input and produce all $2N$ bit decisions simultaneously.¹ The "TransCoder" framework demonstrates that transformers can enhance the reliability of existing ECCs while maintaining manageable computational complexity.²¹

The challenge for a transformer is that the SC decoding function is inherently sequential—the

decision for bit i depends on the decisions for bits $0 \dots i - 1$.¹ However, during teacher forcing, the transformer is provided with the "true" previous decisions, potentially allowing the attention mechanism to learn the complex polarization relationships without an explicit tree walk.¹ Using sinusoidal positional embeddings to represent the hierarchical bit-channel

reliabilities could enable the transformer to scale to $N = 1024$ and beyond, where sequential models currently fail.²⁵

Two-Phase Decomposition for Interleaved Paths

The difficulty of the Class B path is that User U and User V are coupled in a way that requires bottom-up updates.¹ A two-phase decomposition approach could resolve this by breaking the interleaved decode into two single-user decodes¹:

1. **Phase 1:** Decode all User U bits using a single-user decoder, treating the un-decoded User V bits as part of the channel noise.¹
2. **Phase 2:** Decode all User V bits using a single-user decoder, conditioned on the estimates (or true bits during teacher forcing) from Phase 1.¹

Since each phase is a single-user decode, it is fully compatible with the parallel fast_decode trick.¹ The primary issue is that Phase 1 will be suboptimal, as User U often operates above its marginal capacity.¹ However, iterative refinement—re-decoding User U given the User V results—could lead to the joint optimum.¹ During training, teacher forcing would provide the necessary "genie" information to make both phases parallel, while the model learns to handle the dependencies iteratively.¹

Scaling Limitations and Error Accumulation

The drive toward parallelization is not only a matter of training speed but of fundamental performance at large block lengths.¹ Empirical results for neural MAC decoders show a significant block error rate (BLER) gap at $N \geq 256$ on continuous channels like the GMAC.¹

Block Length (N)	SC BLER (Analytical)	NN-SC BLER (Neural)	Ratio (NN/SC)
32	0.046	0.046	1.0x
64	0.025	0.026	1.04x
128	0.016	0.017	1.06x
256	0.005	0.015	3.0x
512	0.001	0.008	8.0x

The degradation at large N is driven by a scaling law where the error ratio grows as $N^{2.6}$.¹

The primary root cause identified is the z-encoder information bottleneck.¹ On continuous channels, the MLP-based z-encoder maps the real-valued channel output to a d -dimensional embedding, effectively quantizing the space and losing information—estimated at ~0.04 bits per symbol for a $d = 16$ model.¹ This information loss accumulates over the $O(N \log N)$ tree operations, eventually corrupting the leaf decisions.¹

The second major cause is sequential error accumulation.¹ Each of the $\sim 6N$ MLP operations in the SC tree walk introduces a small approximation error ϵ .¹ In a purely sequential model, these

errors accumulate as $O(K)$ where K is the number of sequential operations.¹ For $N = 1024$, where $K \approx 6123$, the accumulation factor is approximately 78, making

optimization nearly impossible.¹ Reducing the sequential depth to $O(\log N)$ would drop this factor to ~3.3, theoretically allowing the neural decoder to scale successfully to modern block lengths.¹

Theoretical Support from BEMAC Results

The success of the neural decoder on the Binary Erasure MAC (BEMAC) provides the strongest evidence that the underlying tree operations (CalcLeft, CalcRight, CalcParent) are fundamentally sound.¹ On the BEMAC, the z-encoder is an exact lookup table (nn.Embedding), resulting in zero information loss at the input stage.¹

N (BEMAC)	SC BLER	NN-SC BLER	NN/SC Ratio
64	0.0056	0.003	0.54x
128	0.0020	0.0012	0.60x
256	8e-5	4e-5	0.50x
1024	1e-4	1e-4	1.0x

The neural decoder actually beats the analytical SC decoder on the BEMAC by up to 50%.¹ This is because the neural embeddings can capture more nuanced decision boundaries at the leaf positions than the simple marginalized probabilities used by the analytical decoder.¹ This "beats-analytical" performance proves that if the training depth and information bottleneck issues can be solved for continuous channels, the neural architecture is capable of achieving or exceeding state-of-the-art multi-user performance.¹

Synthesis of a Parallel Training Architecture

The proposed path forward involves a hierarchical, depth-parallel training scheme with auxiliary losses.¹ Instead of training the entire tree walk end-to-end, each depth of the tree should be trained independently, parallel over all $N/2^d$ positions.¹

1. **Depth 0 Training:** Train the z-encoder and the initial CalcLeft/CalcRight MLPs to represent the polarized sub-problems at the root.¹
2. **Intermediate Targets:** Define training targets for intermediate edges based on the analytical properties of CalcParent—specifically that the CalcParent of decided bits equals the XOR of the children's message bits.¹
3. **Level-Specific MLPs:** Use the "freeze and extend" method to add trainable level-specific MLPs for each new depth in a curriculum, maintaining weight-sharing only where it does not compromise expressivity.¹

This depth-parallel approach reduces gradient depth to $O(\log N)$ while maintaining the essential structural priors of the polar code.¹ When combined with a high-capacity z-encoder ($d \geq 32$) and iterative refinement for Class B paths, this synthesis offers the most likely route to a breakthrough in multi-user neural decoding.¹

The computational efficiency of such a parallelized decoder would be transformative. Currently, the neural SC decoder requires ~360x more FLOPs than the analytical version.¹ Parallelization would allow for GPU-based training of multiple codewords simultaneously without the sequential

"tree walk" bottleneck, reducing training times from days to hours and enabling the scaling of neural polar codes to $N = 1024$ and beyond.¹

Implementation and Feasibility Analysis

The feasibility of these approaches is supported by the existence of optimized C++ extensions and the proven success of curriculum learning in training sequential decoders.¹ A custom PyTorch C++ extension has already demonstrated a 1.34x speedup by eliminating Python interpreter overhead during the tree walk.¹ Applying this to a parallel BP or Transformer architecture would likely yield even greater performance gains due to better utilization of SIMD and tensor cores.²⁷

Approach	Expected Complexity	Implementation Difficulty	Success Likelihood
Algebraic (Character)	$O(\log N)$	High (needs WHT design)	High
Tropical (Max-Log)	$O(\log N)$	Moderate	Moderate
Factored BP	$O(I)$	Low	High
Transformer	$O(1)$ per layer	Moderate	Moderate
Two-Phase Refinement	$O(\log N)$	Moderate	Very High

The "Two-Phase Refinement" approach is ranked highest for immediate success because it leverages the existing, proven single-user parallelization (fast_ce) while using iterative refinement to address the joint marginal capacity issue.¹ This strategy allows the decoder to remain "polar-structured" while bypassing the $\mathbb{Z}_2 \times \mathbb{Z}_2$ coupling that makes direct CalcLeft parallelization difficult.¹

In conclusion, the parallelization of training for neural MAC polar decoders is a multi-dimensional challenge that requires moving beyond standard teacher forcing.¹ By integrating group-theoretic decompositions, tropical algebraic approximations, and iterative factor graph updates, it is possible to achieve a $O(\log N)$ training depth.¹ This would not only close the performance gap at large block lengths but would also unlock the full potential of data-driven decoders for complex, multi-user, and unknown channel environments.¹ The convergence of these methods marks a new chapter in the evolution of capacity-achieving neural codes for the next generation of wireless standards.¹⁷

Works cited

1. full_project_report.pdf
2. Polar Coding Tutorial - Simons Institute, accessed April 4, 2026,

- <https://simons.berkeley.edu/sites/default/files/docs/2689/slidesarikan.pdf>
3. Polar code (coding theory) - Wikipedia, accessed April 4, 2026, [https://en.wikipedia.org/wiki/Polar_code_\(coding_theory\)](https://en.wikipedia.org/wiki/Polar_code_(coding_theory))
 4. [PDF] Feedback is Good, Active Feedback is Better: Block Attention Active Feedback Codes | Semantic Scholar, accessed April 4, 2026, <https://www.semanticscholar.org/paper/Feedback-is-Good%2C-Active-Feedback-is-Better%3A-Block-Ozfatura-Shao/984e2a6ff7fa5281e2170e699243c3a75b4ee4ec>
 5. APPROXIMATE BELIEF PROPAGATION DECODER FOR POLAR CODE Menghui Xu, Shusen Jing, Chuan Zhang, Jun Lin, Weikang Qian and Xiaohu You, accessed April 4, 2026, https://gc.sjtu.edu.cn/~wkqian/papers/Xu_Jing_Zhang_Lin_Qian_You_Approximate_Belief_Propagation_Decoder_for_Polar_Code.pdf
 6. Hardware Decoders for Polar Codes: An Overview - arXiv, accessed April 4, 2026, <https://arxiv.org/pdf/1606.00737>
 7. Parallelism versus Latency in Simplified Successive-Cancellation Decoding of Polar Codes, accessed April 4, 2026, <https://par.nsf.gov/servlets/purl/10406735>
 8. A Hybrid Quantum-Classical Approach based on the Hadamard Transform for the Convolutional Layer - arXiv, accessed April 4, 2026, <https://arxiv.org/html/2305.17510v3>
 9. LogSumExp - Wikipedia, accessed April 4, 2026, <https://en.wikipedia.org/wiki/LogSumExp>
 10. LogSumExp and Its Numerical Stability - Lei Mao's Log Book, accessed April 4, 2026, <https://leimao.github.io/blog/LogSumExp/>
 11. Asymmetric Linear-Combination-of-Unitaries Realization of Quantum Convolution via Modular Adders - arXiv, accessed April 4, 2026, <https://arxiv.org/pdf/2603.15214>
 12. An Analytical Study of the Min-Sum Approximation for Polar Codes - arXiv, accessed April 4, 2026, <https://arxiv.org/html/2501.13092v1>
 13. A contemplation of logsumexp - Loren Lugosch, accessed April 4, 2026, <https://lorenlugosch.github.io/posts/2020/06/logsumexp/>
 14. Mathematical Implementation of MAX LOG MAP Algorithm for Low Power Applications in Turbo Decoders, accessed April 4, 2026, https://www.ijareeie.com/upload/2014/may/1_Mathematical.pdf
 15. Improved Belief Propagation List Decoding for Polar Codes - MDPI, accessed April 4, 2026, <https://www.mdpi.com/2079-9292/11/15/2458>
 16. Belief Propagation Decoder - Emergent Mind, accessed April 4, 2026, <https://www.emergentmind.com/topics/belief-propagation-decoder>
 17. Belief Propagation Decoding of Polar Codes on Permuted Factor Graphs - arXiv, accessed April 4, 2026, <https://arxiv.org/pdf/1801.04299>
 18. Learned Belief-Propagation Decoding with Simple Scaling and SNR Adaptation, accessed April 4, 2026, <https://www.semanticscholar.org/paper/Learned-Belief-Propagation-Decoding-with-Simple-and-Lian-Carpi/96d9d10a6203e718d69927643d82e9ed59f8f112>
 19. A Very Fast Joint Detection for Polar-Coded SCMA - ResearchGate, accessed

April 4, 2026,

https://www.researchgate.net/publication/359811495_A_Very_Fast_Joint_Detection_for_Polar-Coded_SCMA

20. A Very Fast Joint Detection for Polar-Coded SCMA - IEEE Xplore, accessed April 4, 2026, <https://ieeexplore.ieee.org/iel7/6287639/6514899/09751596.pdf>
21. (PDF) TransCoder: A Neural-Enhancement Framework for Channel Codes - ResearchGate, accessed April 4, 2026, https://www.researchgate.net/publication/398134683_TransCoder_A_Neural-Enhancement_Framework_for_Channel_Codes
22. TransCoder: A Neural-Enhancement Framework for Channel Codes - arXiv, accessed April 4, 2026, <https://arxiv.org/html/2511.22539v1>
23. Nested Construction of Polar Codes via Transformers - arXiv, accessed April 4, 2026, <https://arxiv.org/html/2401.17188v1>
24. Feedback is Good, Active Feedback is Better: Block Attention Active Feedback Codes | Request PDF - ResearchGate, accessed April 4, 2026, https://www.researchgate.net/publication/374942549_Feedback_is_Good_Active_Feedback_is_Better_Block_Attention_Active_Feedback_Codes
25. The 2023 Conference on Empirical Methods in Natural Language Processing, accessed April 4, 2026, <https://aclanthology.org/events/emnlp-2023/>
26. CRISP: Curriculum based Sequential neural decoders for Polar code family - Proceedings of Machine Learning Research, accessed April 4, 2026, <https://proceedings.mlr.press/v202/hebbbar23a/hebbbar23a.pdf>
27. Parallel FFT and Parallel Cyclic Convolution Algorithms with Regular Structures and no Processor Intercommunication, accessed April 4, 2026, https://archive.ll.mit.edu/HPEC/agendas/proc05/Day_1/Abstracts/1150_Teixeira_A.pdf
28. A 14 ns-Latency 9 Gb/s 0.44 "mm"² 62 pJ/b Short-Blocklength LDPC Decoder ASIC in 22FDX - arXiv, accessed April 4, 2026, <https://arxiv.org/html/2512.17834v1>
29. Past Research Summary - Henry Pfister, accessed April 4, 2026, <http://pfister.ee.duke.edu/research.html>